



ATAJOS UTILES DE VS CODE



Seleccionar siguiente coincidencia

Ctrl + D



Seleccionar todas las coincidencias

Ctrl + Shift + L



Cambiar nombre inteligente

F2



Buscar en archivo

Ctrl + F



Buscar en todo el proyecto

Ctrl + Shift + F



Comentar linea

Ctrl + /



Duplicar linea

Shift + Alt + Abajo



Mover linea

Alt + Arriba / Abajo



Abrir terminal

Ctrl + `



Guardar archivo

Ctrl + S



Formatear codigo

Shift + Alt + F



Abrir paleta de comandos

Ctrl + Shift + P



Tip: usa F2 para variables y funciones; es mas seguro que reemplazar texto.

{ ECMAScript MODERNO }

Forma actual de escribir JavaScript con código más limpio, modular y asíncrono.

1



1. Activar módulos

En package.json:

```
"type": "module"
```

2



2. Exportar

Permite compartir funciones, variables o clases desde un archivo.

```
export function saludar() {}  
export default app
```

3



3. Importar

Permite usar código que viene de otro archivo.

```
import { saludar } from './utils.js'  
import app from './app.js'
```

4



4. Asíncronia

Permite trabajar con tareas que tardan sin bloquear el programa.

```
async function main() {}  
const datos = await obtenerDatos()
```

5



5. Manejo de errores

```
try { await tarea() } catch (error) {}
```



Idea clave

Exportas lo que quieres compartir. Importas lo que quieres usar. Usas async/await cuando una tarea debe esperar una respuesta.

EXPORTAR E IMPORTAR CLASES

ECMAScript usa módulos para compartir código entre archivos.

1. Activar módulos

package.json

```
1 {  
2   "type": "module"  
3 }
```

2. Exportación nombrada

producto.js

```
1 export class Producto {}
```

Se importa con llaves.

```
1 import { Producto }  
2 from './producto.js'
```

3. Exportación por defecto

usuario.js

```
1 export default class Usuario {}
```

Se importa sin llaves.

```
1 import Usuario  
2 from './usuario.js'
```

4. Crear objetos

main.js

```
1 const producto = new Producto()  
2 const usuario = new Usuario()
```

producto.js



usuario.js



Idea clave

Con export compartes una clase. Con import la usas en otro archivo. Usa llaves solo cuando la exportación tiene nombre.

EXPRESS.JS



Framework de Node.js para crear servidores y API REST de forma sencilla.

Instalacion



```
npm install express
```

Activar ECMAScript



package.json

```
"type": "module"
```

Servidor basico



```
1 import express from 'express'
2 const app = express()
3 app.listen(3000)
```

Conceptos clave



app

La aplicacion o servidor.



req

La solicitud del cliente.



res

La respuesta del servidor.

Rutas REST

GET

/productos --->

Leer datos

POST

/productos --->

Crear datos

PUT

/productos/:id -->

Actualizar datos

DELETE

/productos/:id -->

Eliminar datos

JSON

{</>}

```
app.use(express.json())
```

Permite recibir datos en req.body.



Idea clave

Express organiza tu backend con rutas, metodos HTTP, solicitudes y respuestas.

≡ RUTAS POST EN EXPRESS ≡



POST se usa para enviar datos al servidor y crear un recurso nuevo.

Flujo



Cliente envia JSON



Servidor recibe datos



Servidor responde

Middleware necesario

```
app.use(express.json())
```



Permite leer JSON en req.body.

Estructura

```
app.post(PATH, HANDLER)
```

PATH

Ruta donde se crean datos.

HANDLER

Funcion que procesa la solicitud.

req.body



Datos enviados por el cliente.

201 Creado

201

Respuesta recomendada al crear algo.

Ejemplo

```
app.post('/productos', (req, res) => {  
  const producto = req.body  
  res.status(201).json(producto)  
})
```

Idea clave



Con POST el cliente envia datos, Express los lee en req.body y el servidor responde normalmente con 201.



MODULO CRYPTO EN NODE.JS



Sirve para crear datos seguros:
IDs, numeros aleatorios, hashes y tokens.



randomUUID()

Crea un ID unico y seguro.

```
1 crypto.randomUUID()
```



randomInt(min, max)

Crea un numero entero
aleatorio seguro.

```
1 crypto.randomInt(1, 100)
```



randomBytes(size)

Crea bytes aleatorios para
tokens o claves.

```
1 crypto.randomBytes(16)
```



Por que no Math.random()?

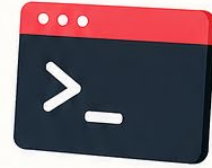
crypto es mejor para seguridad. Math.random()
solo sirve para cosas simples como juegos o pruebas.



Tip: se escribe random, no ramdom.



NPM



Node Package Manager

Gestor de paquetes para proyectos de JavaScript y Node.js.

Para que sirve?



Instalar librerías



Administrar dependencias



Ejecutar scripts del proyecto



Compartir paquetes

Archivos importantes



package.json

Ficha del proyecto: nombre, scripts y dependencias.



package-lock.json

Guarda versiones exactas instaladas.



node_modules/

Carpeta donde viven los paquetes instalados.

Comandos básicos

```
$ npm init -y
```

Crea package.json rápido.

```
$ npm install express
```

Instala un paquete.

```
$ npm install
```

Instala todas las dependencias.

```
$ npm run start
```

Ejecuta un script.



Idea clave

NPM te ayuda a traer código externo, organizar tu proyecto y ejecutar comandos fácilmente.

⚡ RUTAS EN EXPRESS ⚡



Una ruta define como una aplicacion responde a una solicitud del cliente.



Forma basica

```
app.METHOD(PATH, HANDLER)
```



app

Instancia de Express.



METHOD

Metodo HTTP: GET, POST, PUT, PATCH o DELETE.



PATH

Ruta o endpoint en el servidor.



HANDLER

Funcion que se ejecuta cuando la ruta coincide.

Ejemplo

```
1 app.get('/productos', (req, res) => {})
```



req

Solicitud del cliente.



res

Respuesta del servidor.



Idea clave

Las rutas conectan un metodo HTTP con una URL y una funcion que responde al cliente.